Trabalho 2 — Comparando Funções Hash

Disciplina: Resolução de Problemas Estruturados em Computação | Turma: B
João Pedro Gadens Mosson – Matrícula: 40107924
João Pedro Magri Pozzan – Matrícula: 40109973
Eduardo Skoroboatei Gomes – Matrícula: 40112476
Renan Cassou Rodrigues – Matrícula: 40106457
Data: 06/11/2026

1. Implementação

O programa foi desenvolvido em um único arquivo (Trabalho2.java), com base nas assinaturas, fórmulas e algoritmos definidos na especificação. As quatro funções hash foram implementadas conforme exigido:

- **Divisão:** obtém os três últimos dígitos da chave por meio de chave % 1000.
- **Primo:** calcula chave % 997, com um retorno no intervalo de 0 a 996.
- **Dobra:** separa a chave em três partes (chave % 1000, (chave / 1000) % 1000 e chave / 1000000), soma as partes e retorna o resto da divisão por 1000.
- **Meio do Quadrado (hashMeioQuadrado):** realiza a conversão para long, calcula chave * chave, extrai os dígitos das posições 6, 7 e 8.

O tratamento de colisões seguiu o rehashing. As operações de inserção e busca registraram as métricas de acessos e colisões. A geração das chaves de cada conjunto utilizou java.util.Random instanciado com a seed 1990 a cada novo conjunto, descartando as duplicatas.

Dificuldades

A principal dificuldade não foi na lógica ou nos algoritmos em si, mas em seguir às restrições de funcionalidades permitidas e proibidas impostas pelo trabalho:

- **Ausência de break e continue:** Exigiu manter variáveis dedicadas a manter o resultado obtido dentro do loop e checá-las novamente após o fim do loop.
- **Proibição de switch e for-each:** Fez com que fosse preciso utilizar muitos blocos if / else e controlar índices manualmente via contadores.

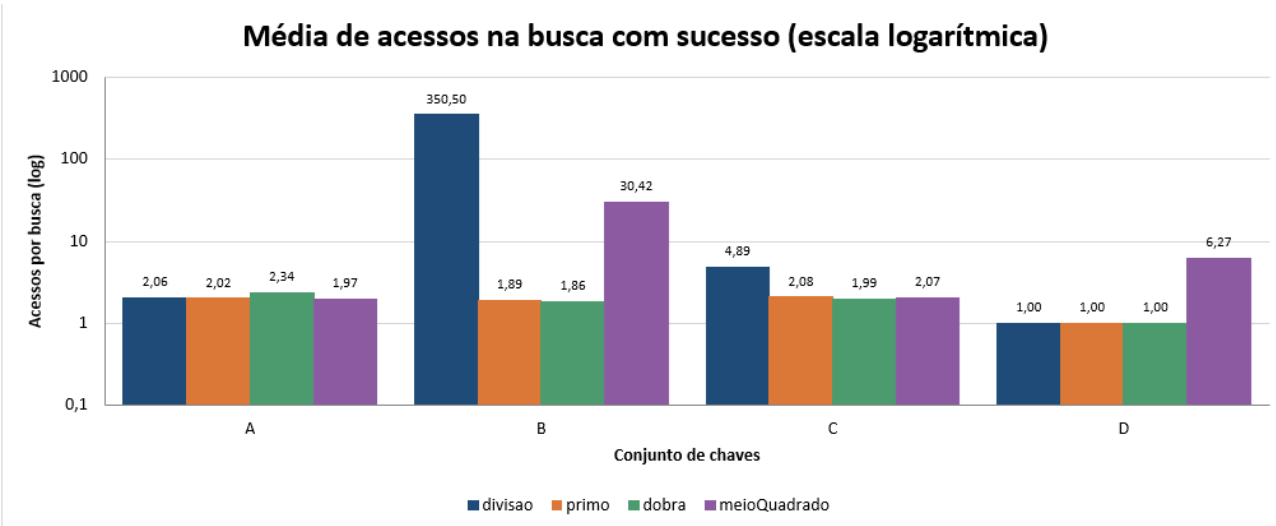
Resultado do programa: TESTES: 17 de 17 aprovados

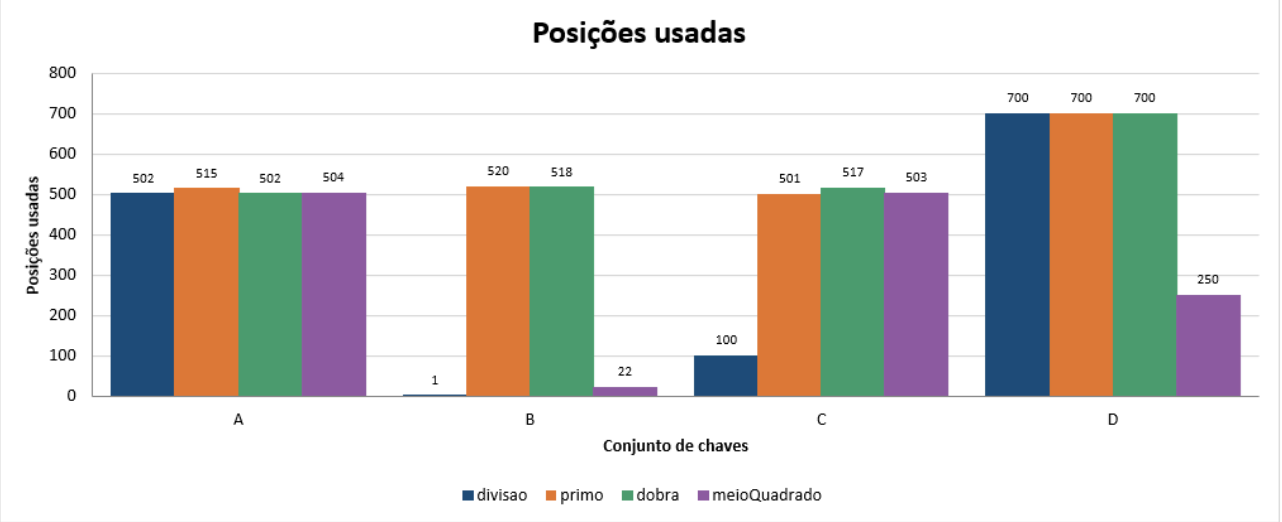
2. Tabela de resultados

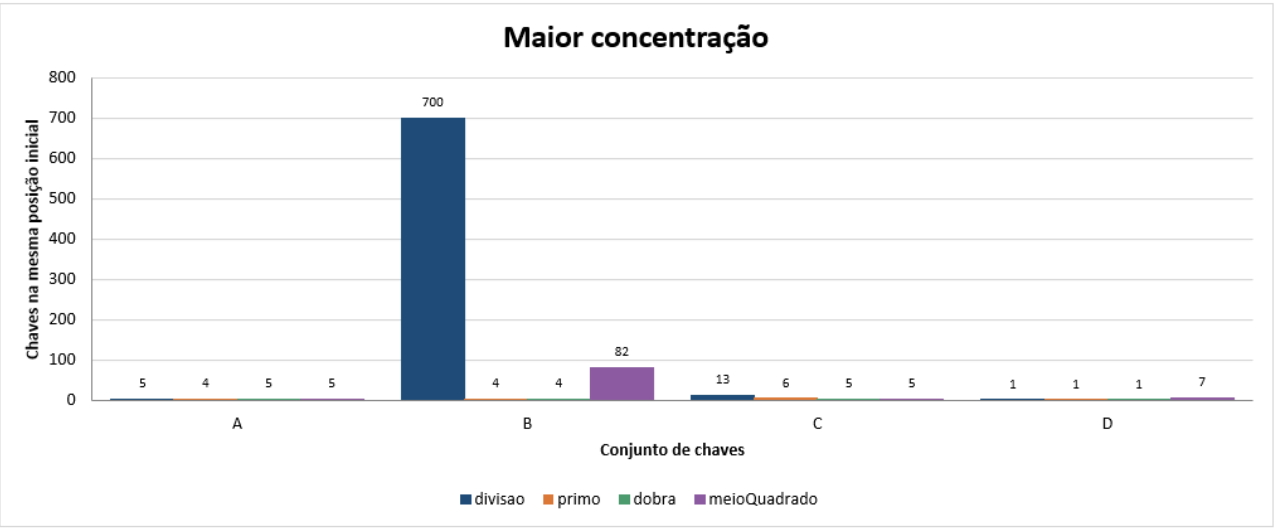
conjunto	funcao	chaves	posicoesUsadas	posicoesNuncaUsadas	maiorConcentracao	somaQuadrados	colisoes	acessosInsercaoTotal	maiorAcessosUmaInsercao	acessosBuscaSucessoMedia	tempoInsercaoNsPorOp
A	divisao	700	502	498	5	1212	253	1439	17	2.06	312
A	primo	700	515	485	4	1150	240	1413	26	2.02	154
A	dobra	700	502	498	5	1198	250	1641	40	2.34	209
A	meioQuadrado	700	504	496	5	1204	238	1377	17	1.97	172

B	divisao	700	1	999	700	490000	699	245350	700	350.50	6233
B	primo	700	520	480	4	1136	216	1325	27	1.89	151
B	dobra	700	518	482	4	1120	233	1302	20	1.86	187
B	meioQuadrado	700	22	978	82	27118	678	21291	225	30.42	703
C	divisao	700	100	900	13	5740	600	3425	36	4.89	215
C	primo	700	501	499	6	1210	245	1459	29	2.08	154
C	dobra	700	517	483	5	1154	226	1393	35	1.99	157
C	meioQuadrado	700	503	497	5	1208	242	1447	40	2.07	137
D	divisao	700	700	300	1	700	0	700	1	1.00	109
D	primo	700	700	300	1	700	0	700	1	1.00	107
D	dobra	700	700	300	1	700	0	700	1	1.00	116
D	meioQuadrado	700	250	750	7	2822	650	4392	10	6.27	165

3. Gráficos







4. Respostas às perguntas do item 7.2

1. No conjunto A (aleatório), as quatro funções tiveram desempenho parecido? Qual teve menos colisões e por que a diferença entre elas é pequena?

R: Sim, o desempenho foi parecido: as colisões variaram de 238 a 253, e a busca média, de 1,97 a 2,34 acessos. A *meioQuadrado* teve menos colisões (238). A diferença é pequena porque chaves aleatórias não têm um padrão que favoreça ou prejudique uma função específica, então todas as espalham de forma próxima ao acaso.

2. No conjunto B, o que aconteceu com a função divisão? Explique por que posicoesUsadas foi 1 e quantos acessos gastou a inserção da última chave.

R: A função divisão teve o pior caso possível: como todas as chaves de B são múltiplos de 1000, chave % 1000 sempre dá 0. Por isso posicoesUsadas foi 1, já que a métrica conta apenas as posições iniciais calculadas pelo hash. Com o rehash linear, as posições 0 a 699 foram ocupadas, e a tabela virou na prática uma lista sequencial. A inserção da última chave precisou de 700 acessos: 699 posições ocupadas e mais 1 posição livre. O total de acessos foi 245350 e a busca média, 350,50.

3. Ainda no conjunto B, por que a função primo se saiu tão melhor que a divisão? Relacione com o fato de 1000 = $2^3 \times 5^3$ e de 997 ser primo.

R: Todas as chaves de B são múltiplos de 1000 (1000 = $2^3 \times 5^3$), então chave % 1000 é sempre 0 e a divisão manda as 700 chaves para a posição 0, usando só 1 posição e chegando a 350,50 acessos por busca. Como 997 é primo e não tem fator em comum com 1000, o resto por 997 espalha bem as chaves: o primo usou 520 posições e teve busca média de 1,89, embora ainda haja colisões (maiorConcentracao 4).

4. No conjunto C, por que a função divisão usa apenas 100 posições? Que fração da tabela ficou sem nenhuma chave vinda da função hash?

R: As chaves de C terminam em 0, então chave % 1000 só pode valer 0, 10, 20, ..., 990. Como existem apenas 100 posições assim, posicoesUsadas = 100 e sobraram 900 posições sem chave, ou seja, 90% da tabela. As 700 chaves se concentraram nessas 100 posições (maiorConcentracao 13 e 600 colisões), e a busca média foi 4,89 contra cerca de 2,0 nas outras funções.

5. No conjunto D (sequencial), a divisão, o primo e a dobra não tiveram nenhuma colisão. Explique por quê. Por que o meio do quadrado foi pior? Cite posicoesUsadas e acessosBuscaSucessoMedia.

R: As chaves de D são 1.000.000 + k, com k de 1 a 700. Como são 700 números consecutivos, as três funções geram posições todas distintas: de 1 a 700 na divisão, de 10 a 709 no primo e de 2 a 701 na dobra. Por isso têm 0 colisões, posicoesUsadas = 700 e acessosBuscaSucessoMedia = 1,00. No meio do quadrado, $(10^6 + k)^2 = 10^{12} + 2k \cdot 10^6 + k^2$, e os dígitos 6 a 8 dependem sobretudo de $20 \cdot k \bmod 1000$, que só assume 50 valores (mais um acréscimo de 0 a 4 vindo de k^2). Isso limita a 250 posições, com busca média de 6,27.

6. Explique por que posicoesNuncaUsadas da função primo é sempre pelo menos 3.

R: A função primo devolve chave % 997, que vai de 0 a 996. As posições 997, 998 e 999 nunca são calculadas pela função, só podem ser ocupadas por rehashing. Por isso posicoesNuncaUsadas é sempre pelo menos 1000 − 997 = 3. Na prática o valor é bem maior: no conjunto D, em que o primo usou 700 posições, posicoesNuncaUsadas foi 300.

7. Mostre, com a conta, por que a função do meio do quadrado precisa de long para a chave 9999999. Qual é o maior valor que cabe em um int em Java?

R: O maior valor que cabe em um int no Java é 2.147.483.647 ($2^{31} - 1$), pois ele utiliza 32 bits com sinal e qualquer valor acima causaria overflow. Na função do meio do quadrado, a chave é multiplicada por si mesma (9.999.999 * 9.999.999 = 99.999.980.000.001), e como 99.999.980.000.001 > 2.147.483.647, o resultado ultrapassa o limite do int, tornando necessário o uso de long (que suporta até $9,22 \times 10^{18}$) para evitar a perda de dados.

8. Em qual conjunto foi maior a diferença entre a melhor e a pior função? Compare com acessosBuscaSucessoMedia e com somaQuadrados e explique o que somaQuadrados mede.

R: A maior diferença foi no conjunto B. Na busca, a dobra teve 1,86 e a divisão 350,50, cerca de 188 vezes mais. No somaQuadrados, foram 1120 contra 490000. Nos outros conjuntos a diferença foi menor: A de 1,97 a 2,34, C de 1,99 a 4,89 e D de 1,00 a 6,27. O somaQuadrados soma, nas 1000 posições, o quadrado da quantidade de chaves que a função mandou para cada uma. Ele mede a concentração, de 700 (cada chave sozinha) a 490000 (todas juntas).

9. Por que maiorAcessosUmaInsercao pode ser maior que maiorConcentracao? Cite uma linha do CSV em que isso acontece.

R: Isso acontece porque as duas métricas medem situações diferentes. maiorConcentracao é a maior quantidade de chaves com a mesma posição hash inicial. Já maiorAcessosUmaInsercao conta quantas posições foram examinadas em uma única inserção, incluindo os deslocamentos do rehashing linear. Assim, uma chave pode percorrer várias posições ocupadas por colisões anteriores, mesmo que poucas chaves tenham o mesmo hash inicial. Por exemplo, no conjunto A com a função dobra, maiorConcentracao foi 5, enquanto maiorAcessosUmaInsercao chegou a 40.

10. Se os códigos dos produtos do comerciante dos slides terminassem sempre em 00 (por exemplo, 1234500), qual função o grupo recomendaria e por quê?

R: Recomendaria a dobra. Com códigos terminando em 00, a divisão só alcançaria 10 posições (0, 100, ..., 900), o mesmo problema dos conjuntos B e C, em que usou 1 e 100 posições e teve busca média de 350,50 e 4,89. A dobra também soma as outras partes da chave, que variam livremente, e espalha bem: usou 518 posições no B e 517 no C, com busca média de 1,86 e 1,99.

5. Conclusão

R: O grupo escolheria a função dobra como padrão, pois apresentou bons resultados na maioria dos conjuntos: teve a melhor busca média no B (1,86) e no C (1,99), empatou no D (1,00) e ficou só um pouco acima das demais no A (2,34). Porém, a função com o melhor resultado varia conforme as chaves utilizadas. A divisão, por exemplo, depende dos últimos dígitos e chegou a 350,50 acessos no B, e o meio do quadrado chegou a 6,27 no D. O primo pode ser melhor quando for importante limitar o pior caso de uma inserção (no máximo 29 acessos, contra 40 da dobra) ou quando os números possuem padrões que prejudicam a dobra.

6. Declaração

Declaramos que o código e o texto deste trabalho foram produzidos pelos integrantes do grupo, sem uso de ferramenta de inteligência artificial generativa e sem cópia de outros grupos ou de fontes externas.

João Pedro Gadens Mosson – Matrícula: 40107924

João Pedro Magri Pozzan – Matrícula: 40109973

Eduardo Skoroboatei Gomes – Matrícula: 40112476

Renan Cassou Rodrigues – Matrícula: 40106457

7. Referências

CABRAL, Andrey. **09 - Tabela Hash**. Slides da disciplina Resolução de Problemas Estruturados em Computação. PUCPR, 2026.

CABRAL, Andrey. **Trabalho 2: Comparando Funções Hash**. Enunciado da atividade, Resolução de Problemas Estruturados em Computação. PUCPR, 2026.